Informe final Etapa 2 · Rumbo Sur

Nov 12, 2026 · @valentin

1. Introducción

En la Etapa 2 implementamos en Java 17 el sistema Rumbo Sur a partir del modelo de clases final de la Etapa 1. El sistema funciona completo: tiene menús de consola para el vendedor (7 opciones), el coordinador (6) y el administrador (5, con el catálogo de tours), una ventana Swing para crear reservas y guarda los datos en un archivo. Hay 168 pruebas unitarias con JUnit 5 y un plan de pruebas funcionales por integrante.

El código está organizado en paquetes según la arquitectura Modelo-Vista-Controlador:

Paquete	Contenido
`modelo`	Entidades del negocio (Tour, Salida, Reserva, Pago, etc.) y enumeraciones
`servicio`	`CalculadoraPrecio` y las políticas de cancelación
`controlador`	`ControladorReservas`, `ControladorSalidas`, `ControladorReportes` y `ControladorCatalogo`
`vista`	Menús de consola (`VistaVendedor`, `VistaCoordinador`, `VistaAdministrador`, `VistaCatalogo`), `EntradaConsola` y la ventana `VentanaReserva`
`persistencia`	`Repositorio`, que guarda y carga los datos
`excepciones`	Excepciones propias del negocio

Para ejecutarlo se corre `Principal.java`. La primera vez carga datos de prueba (`DatosIniciales`): tres tours con su itinerario y su tarifa, tres salidas con guía y vehículo, tres clientes, dos vendedores y una reserva confirmada de ejemplo.

2. Cambios respecto del diseño

Respetamos la estructura del modelo final: los tres controladores, `CalculadoraPrecio` (SRP), la interfaz `PoliticaCancelacion` (OCP y DIP), `Cliente` como creador de la reserva y `Salida` como experta en el cupo. Al programar hubo que ajustar algunos métodos y agregar clases que el diagrama no mostraba.

Cambios en clases del modelo

Clase	En el diseño	En la implementación	Motivo
`Cliente`	`crearReserva(salida, vendedor, pasajeros)`	`crearReserva(salida, vendedor)`; los pasajeros se agregan después con `agregarPasajero`	Cada pasajero se valida por separado (edad mínima, repetidos)
`ControladorReservas`	Crea la reserva y después valida	Controla cupo, edad y pasajeros repetidos **antes** de crear la reserva	Si fallaba una validación quedaba una reserva a medio armar (error C4 del plan de pruebas)
`Reserva`	`confirmar(): void`	`confirmar(): boolean`	Devuelve si se pudo confirmar, según la seña mínima del 30 %
`Reserva`	`reemplazarPasajero(actual, nuevo)`	Recibe también la fecha	Hay que controlar que falten al menos 3 días para la salida
`Reserva`	Sin estos métodos	`motivoAnulacion`, `calcularSenaMinima()`, `calcularSaldo()`, `ocupaCupo()` y `finalizar()`	Saber si la reserva ocupa lugar en la salida y cerrarla cuando la salida termina
`Cancelacion`	Calcula la devolución delegando en la política	Solo registra el importe devuelto; el controlador le pide la devolución a la `PoliticaCancelacion`	Bajo acoplamiento: la cancelación no depende de las políticas
`Pago`	Importe, cuotas y recargo	Agrega la lista de `AjustePago` y `getImporteNeto()`	Los ajustes modifican lo pagado sin cambiar el pago original
`TipoPasajero`	Enumeración simple	Enumeración con el atributo `porcentajeTarifa`	El porcentaje del menor queda en un solo lugar

Clases nuevas

- **Vista:** `MenuPrincipal`, `VistaVendedor`, `VistaCoordinador`, `VistaAdministrador`, `VentanaReserva` (Swing) y `EntradaConsola`, que valida todo lo que se escribe por teclado.
- **Persistencia:** `Repositorio`, que serializa los datos en `rumbosur.dat`.
- **Arranque:** `Principal`, que crea y conecta los objetos, y `DatosIniciales`, que carga los datos de prueba.
- **Excepciones:** `CupoInsuficienteException`, `PasajeroNoHabilitadoException`, `ReservaNoCancelableException`, `PagoInvalidoException`, `GuiaNoDisponibleException`, `VehiculoSinCapacidadException` y `TransicionEstadoInvalidaException`.

Cambios al revisar el alcance

Al terminar comparamos el sistema con la consigna. Completamos los casos de uso mínimos que no se podían usar desde el programa y sacamos lo que la consigna no pedía, para que el código sea más fácil de explicar (pasó de unas 6.380 a 5.624 líneas).

Cambio	Detalle
Se agregó el catálogo de tours (CU01 y CU02)	`ControladorCatalogo` y `VistaCatalogo`: alta y modificación de tours, actividades del itinerario y tarifas por temporada que no se superponen
CU03, CU14 y CU17 en el menú del coordinador	`ControladorSalidas.programarSalida`, asignación de guía por legajo y de vehículo por patente, y cierre de la salida con la liquidación del guía
Se quitó CU10 «Modificar reserva»	La modificación queda cubierta por CU07 (adicionales) y CU11 (reemplazo de pasajero)
Se quitó el historial de la reserva	El seguimiento queda en los pagos, ajustes y cancelaciones, que no se borran (supuesto S30 actualizado)
`Proveedor`	Queda asociado a `Actividad` (0..1): una actividad puede prestarla un proveedor externo
Cupo mínimo	Se cancela si faltan 7 días o menos y el control se hace solo al iniciar el programa
Fin de la salida	Las reservas confirmadas pasan a FINALIZADA

Cambio	Detalle
Menús reducidos	Vendedor 7 opciones (el cliente y los pasajeros se registran al crear la reserva y la reserva se confirma sola al pagar la seña), coordinador 6 y administrador 5
Código sin uso	Se sacaron métodos que nadie llamaba y la exportación de reportes a archivo; `DatosIniciales` y `Principal` quedaron más cortos
Persistencia	Si `rumbosur.dat` está dañado se guarda una copia antes de seguir, y el guardado escribe primero un archivo temporal

Todos estos cambios están en el diagrama de clases final (`docs/uml/06_clases_final_etapa2`) y en el diagrama de casos de uso de la Etapa 2 (`docs/etapa-2/01_casos_de_uso_etapa2`). Los diagramas de secuencia de CU05 y CU12 no cambiaron.

3. Conceptos de la materia aplicados

Cada tema de la cursada quedó aplicado en una parte concreta del código:

Tema	Dónde se aplicó
Herencia y polimorfismo	`Tour` con `TourAventura`, `TourCultural` y `TourNaturaleza`; `Empleado` con `Vendedor`, `Guia`, `Coordinador` y `Administrador`
Interfaces	`PoliticaCancelacion`, con una implementación para el cliente y otra para la agencia; `Comparable` en `Salida`
Datos enumerados	`EstadoReserva`, `EstadoSalida`, `MedioPago`, `TipoPromocion` y `TipoPasajero` (con atributo)
SOLID	SRP en `CalculadoraPrecio`; OCP y DIP en `PoliticaCancelacion`
GRASP	Controlador, Creador (`Cliente`), Experto (`Salida`), Bajo acoplamiento, Polimorfismo y Fabricación pura (`Repositorio`)

Tema	Dónde se aplicó
Colecciones	`List` para pasajeros y pagos, `Map` en los controladores para buscar por código o documento, `Set` para detectar pasajeros repetidos, `Iterator` para quitar adicionales y `Comparator` en los reportes
Excepciones	Siete excepciones verificadas propias, `try/catch/finally` y captura de `InputMismatchException` al leer el teclado
Archivos	Serialización con `Serializable` en `Repositorio`; exportación de la lista de pasajeros con `BufferedWriter`
MVC	Las vistas solo leen y muestran; las reglas están en el modelo y los controladores
Swing	`VentanaReserva` con `JFrame`, `JComboBox`, `JTextField` y un `ActionListener` escrito con lambda
Pruebas unitarias	JUnit 5 con Maven, incluidas pruebas parametrizadas

4. Pruebas

Las 168 pruebas unitarias pasan. Además, cada integrante ejecutó un plan de pruebas funcionales sobre su parte, con entrada, resultado esperado y resultado obtenido.

Plan	Responsable	Qué cubre
`docs/plan-pruebas-valen.md`	Valentín	CU04 a CU07, CU09 y CU11 (clientes, reservas, precios, adicionales, confirmación y reemplazo de pasajeros), más pruebas desde el menú y la ventana Swing, y los casos de uso agregados al revisar el alcance (CU01 a CU03, CU14 y CU17)
`docs/plan-pruebas-fran.md`	Francisco	CU08, CU12, CU13, CU17, CU18 y CU19 (pagos, cancelaciones, vencimientos, liquidaciones y reportes)

Plan	Responsable	Qué cubre
`docs/plan-pruebas-german.md`	Germán	Módulo de operaciones: salidas, ocupación, listas de pasajeros, inicio y fin de salidas y cupo mínimo

Error encontrado y corregido: en el caso C4, al crear una reserva con un pasajero repetido, el sistema la rechazaba pero dejaba una reserva a medio armar en la lista del cliente. Se corrigió controlando los repetidos con un `HashSet` antes de crear la reserva, y se agregó una prueba unitaria para ese caso.

5. Distribución de tareas

Dividimos el trabajo por módulo, en tres días: el día 1 el modelo, el día 2 los controladores y el día 3 las vistas y la integración. Cada uno trabajó en su rama y la sumó a `main` con un pull request.

Integrante	Modelo	Controladores y servicios	Vista y otros
Valentín Umbides	`Reserva`, `Cliente`, `Pasajero`, `Promocion`, `ServicioAdicional`, `AdicionalContratado`, `EstadoReserva`	`ControladorReservas` (CU04 a CU07, CU09 y CU11) y `CalculadoraPrecio`	`MenuPrincipal`, `VistaVendedor`, `VentanaReserva` (Swing), `EntradaConsola`, `Principal`; ControladorCatalogo y VistaCatalogo (CU01 y CU02); plan de pruebas, corrección del error C4 y simplificación final del código
Francisco	`Pago`, `AjustePago`, `Cancelacion`, `Liquidacion`, excepciones propias	Pagos, cancelaciones y vencimientos en `ControladorReservas` (CU08, CU12, CU13); políticas de cancelación; `ControladorReportes`	`Repositorio` (persistencia), modificar tour, `VistaAdministrador`, `pom.xml`; plan de pruebas

Integrante	Modelo	Controladores y servicios	Vista y otros
Germán Rocha	`Tour` y sus subclases, `Salida`, `Tarifa`, `Actividad`, `Proveedor`, `Empleado` y sus subclases, `Vehiculo`, enumeraciones	`ControladorSalidas` (salidas, guías, vehículos, listas de pasajeros, cupo mínimo)	`VistaCoordinador`, `DatosIniciales`, diagramas de la Etapa 2 (secuencia, casos de uso y clases), README; plan de pruebas

Cada uno escribió las pruebas unitarias de sus propias clases.

6. Dificultades

Dificultad	Cómo la resolvimos
Las partes dependían entre sí: los pagos necesitaban `Reserva` y las reservas necesitaban `Salida` y `Tour`	Ordenamos el trabajo por días y marcamos qué tarea esperaba a otra; el día 1 se hizo primero el modelo
Trabajar los tres sobre el mismo repositorio	Una rama por tarea y pull requests a `main`; cuando una rama quedó desactualizada se trajeron los cambios de `main` antes de unirla
Una reserva rechazada quedaba a medio armar (caso C4)	Las validaciones se pasaron antes de crear la reserva y se agregó una prueba unitaria
Al tener dos `Scanner` sobre el teclado, cerrar uno dejaba de leer el otro	Todo el programa usa un solo `Scanner`, que comparten las tres vistas a través de `EntradaConsola`
Después de cargar el archivo, la numeración de reservas volvía a empezar	Al cargar los datos se actualiza el contador estático de `Reserva` con el último número guardado
Configurar JUnit en IntelliJ y en VS Code	Se agregó un `pom.xml` de Maven y en el README se explicó cómo abrir el proyecto y correr las pruebas
Mantener los diagramas iguales al código	Al terminar la implementación se rehicieron en draw.io los diagramas de secuencia de CU05 y CU12 y el diagrama de clases final

El alcance creció más de lo necesario. Al final el código tenía funciones que la consigna no pedía y faltaban opciones de menú para casos de uso mínimos. Lo resolvimos releyendo los casos de uso mínimos y las reglas del enunciado: completamos lo que faltaba (catálogo, programar salida, asignar guía y vehículo, liquidar) y sacamos lo que sobraba, revisando entre los tres cada recorte antes de subirlo.

7. Reflexión final

Lo más valioso fue comprobar que el diseño de la Etapa 1 sirvió. Como el cálculo de precios estaba separado en `CalculadoraPrecio` y las devoluciones en `PoliticaCancelacion`, cada integrante pudo programar su parte sin tocar las clases de los demás. Los cambios que hubo que hacer fueron detalles de métodos, no de estructura.

También aprendimos que las pruebas encuentran errores que no se ven al programar: el caso C4 parecía funcionar porque rechazaba la reserva, pero dejaba datos inconsistentes. Escribir las pruebas con el resultado esperado antes de ejecutarlas nos obligó a revisar cada regla del enunciado.

En cuanto al trabajo en equipo, dividir por módulos y por días, con ramas separadas, nos permitió avanzar en paralelo.

La última revisión nos enseñó a distinguir lo obligatorio de lo opcional: un sistema más chico, que cumple todos los requisitos y se puede explicar entero, vale más que uno con funciones de más. También comprobamos la ventaja de tener pruebas: cada vez que sacamos o cambiamos algo, las pruebas unitarias nos confirmaron que el resto seguía funcionando.